

Informe de Laboratorio 05

Tema: Base de datos

Nota

Estudiante	Escuela	Asignatura
Gustavo Linares Aquino glinares@unsa.edu.pe Geisel Reymar Pacheco Medina gpachecome@unsa.edu.pe Jesus Francisco Silva Pino jsilva@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Desarrollo de Aplicaciones Web Semestre: III Código: 2025000

Laboratorio	Tema	Duración
05	Base de datos	24 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - A	Del 20 Mayo 2026	Al 21 Mayo 2026

Índice

1. Introducción y Objetivos	3
2. Modelo Lógico y Diseño de Entidades (DER)	3
3. Modelo Físico y Código SQL (DDL)	4
4. Despliegue en Supabase e Inserción de Datos (Seeding)	5
4.1. Creación de Tablas vía SQL Editor	5
4.2. Inserción de Datos Coherentes con Bloques Anónimos (PL/pgSQL)	6
5. Configuración de Seguridad: Row Level Security (RLS)	8
6. Integración y Pruebas en Postman	8
6.1. Configuración del Entorno de Trabajo (Environments)	8
6.2. Inclusión Obligatoria de Cabeceras (Headers)	9
6.3. Validación de Respuestas Exitosas	9
7. Conclusiones	13

8. Bibliografía y Referencias de Profundización	13
9. Repositorio del Proyecto	14
10. Rúbricas	14
10.1. Entregable Informe	14
10.2. Rúbrica para el contenido del Informe y demostración	15

1. Introducción y Objetivos

El presente informe detalla el proceso técnico seguido para el diseño, despliegue y validación de una base de datos relacional orientada a un sistema de matrículas académico (enrollments). La solución utiliza un enfoque moderno de Backend-as-a-Service (BaaS) mediante Supabase, aprovechando la potencia del motor relacional PostgreSQL. La validación del acceso a los datos y la verificación de la exposición de la API REST se realizan usando la herramienta Postman.

Objetivos del Laboratorio:

- Diseñar un Modelo Entidad-Relación (DER) que cumpla con los estándares técnicos y convenciones de nombres del curso.
- Implementar el modelo físico en PostgreSQL utilizando identificadores universales únicos (UUID) y esquemas estandarizados de auditoría.
- Desplegar y administrar el esquema relacional en la plataforma Supabase.
- Configurar políticas de seguridad de nivel de fila (Row Level Security - RLS) para el control de accesos públicos y privados.
- Validar las operaciones de consulta (GET) y manipulación de datos (POST/PATCH) consumiendo la API autogenerada a través de Postman, resolviendo fallas comunes de conectividad, enrutamiento y autenticación.

2. Modelo Lógico y Diseño de Entidades (DER)

Para cumplir con las pautas de diseño arquitectónico establecidas en la guía del laboratorio, se definieron cuatro entidades principales estructuradas bajo reglas estrictas: nombres de tablas en inglés y plural, atributos en nomenclatura camelCase, y nomenclatura de tablas intermedias en orden alfabético estricto.

Las entidades que componen el ecosistema son:

- **users**: Entidad central encargada de gestionar las credenciales y perfiles base que se vinculan directamente con los procesos de auditoría del sistema.
- **students**: Almacena los registros detallados de los estudiantes inscritos, manteniendo llaves foráneas hacia el usuario correspondiente.
- **courses**: Catálogo de asignaturas disponibles dentro del sistema de matrícula.
- **courses_students**: Tabla asociativa que rompe la relación de muchos a muchos (N:M) existente entre cursos y estudiantes. Sigue la regla de orden alfabético estricto (courses precede a students).

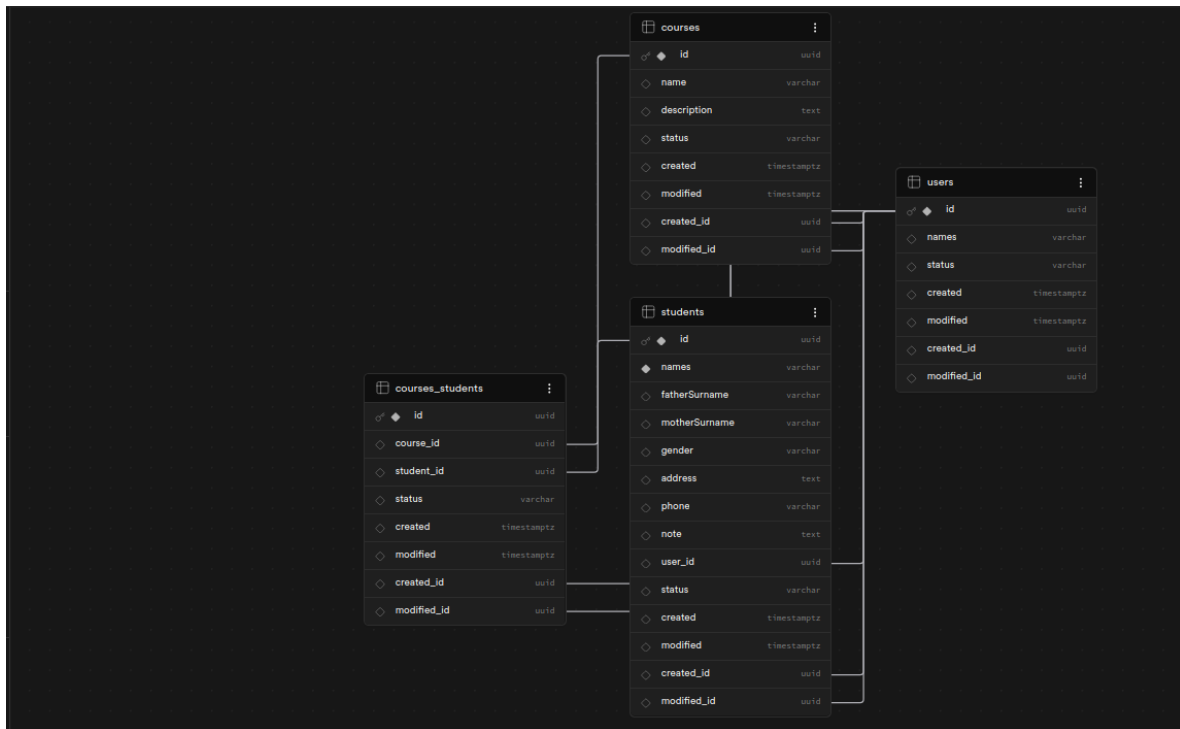


Figura 1: Modelo de Entidad-Relación (DER)

3. Modelo Físico y Código SQL (DDL)

A continuación, se presenta el código SQL de definición de datos (DDL) utilizado para materializar las tablas en PostgreSQL. Se aplican campos de auditoría estándar obligatorios en cada entidad: id (UUID auto-generado), status (para borrado lógico), created, modified, created_id y modified_id.

```
-- 1. TABLA: users (Base de autenticación y auditoría)
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    names VARCHAR(100) NOT NULL,
    status VARCHAR(20) DEFAULT 'ACTIVE',
    created TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    modified TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    created_id UUID,
    modified_id UUID
);

-- 2. TABLA: students (Detalles personales de alumnos)
CREATE TABLE students (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    names VARCHAR(100) NOT NULL,
    "fatherSurname" VARCHAR(100),
    "motherSurname" VARCHAR(100),
    gender VARCHAR(20),
    address TEXT,
```

```
phone VARCHAR(20),
note TEXT,
user_id UUID REFERENCES users(id),
status VARCHAR(20) DEFAULT 'ACTIVE',
created TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
modified TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
created_id UUID REFERENCES users(id),
modified_id UUID REFERENCES users (id)
);

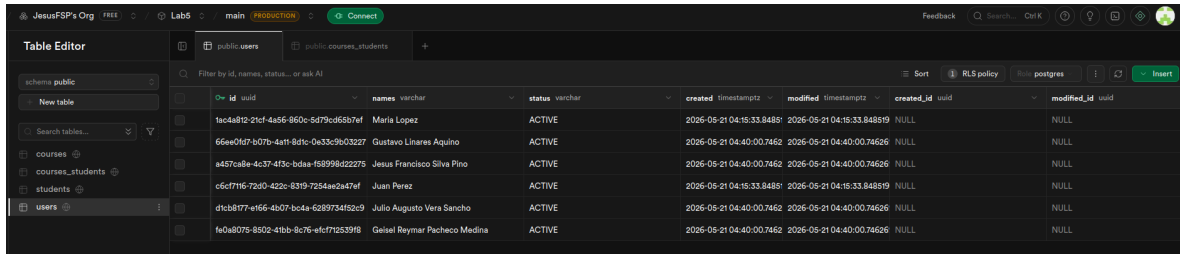
-- 3. TABLA: courses (Catlogo de asignaturas)
CREATE TABLE courses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(100) NOT NULL,
  description TEXT,
  status VARCHAR(20) DEFAULT 'ACTIVE',
  created TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  modified TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  created_id UUID REFERENCES users(id),
  modified_id UUID REFERENCES users(id)
);

-- 4. TABLA INTERMEDIA: courses_students (Relacin N:M)
CREATE TABLE courses_students (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  course_id UUID REFERENCES courses (id) ON DELETE CASCADE,
  student_id UUID REFERENCES students(id) ON DELETE CASCADE,
  status VARCHAR(20) DEFAULT 'ACTIVE',
  created TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  modified TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  created_id UUID REFERENCES users(id),
  modified_id UUID REFERENCES users (id)
);
```

4. Despliegue en Supabase e Inserción de Datos (Seeding)

4.1. Creación de Tablas vía SQL Editor

El script de inicialización se ejecutó en la consola web de Supabase accediendo al menú lateral "SQL Editor", creando una nueva consulta (New Query) y presionando Run. Esto estructuró de inmediato el esquema de base de datos relacional.



id	names	status	created	modified	created_id	modified_id
fac4a812-21cf-4a56-850c-5d79c65b7ef	Maria Lopez	ACTIVE	2026-05-21 04:16:33.848819	2026-05-21 04:16:33.848819	NULL	NULL
66ee0d67-b07b-4a7f-8d7c-0a33c9b03227	Gustavo Linares Aquino	ACTIVE	2026-05-21 04:40:00.7462	2026-05-21 04:40:00.74626	NULL	NULL
8457ca8e-4c37-473c-bda5-f58998d22275	Jesus Francisco Silva Pino	ACTIVE	2026-05-21 04:40:00.7462	2026-05-21 04:40:00.74626	NULL	NULL
c6cf7f16-72d0-422c-6319-7254ae2a47ef	Juan Perez	ACTIVE	2026-05-21 04:16:33.848819	2026-05-21 04:16:33.848819	NULL	NULL
d1cb8777-e166-4b07-b04a-6289734f52c9	Julio Augusto Vera Sancho	ACTIVE	2026-05-21 04:40:00.7462	2026-05-21 04:40:00.74626	NULL	NULL
fa0a8075-8502-41b5-8c76-efc772539f8	Geisel Reynar Pacheco Medina	ACTIVE	2026-05-21 04:40:00.7462	2026-05-21 04:40:00.74626	NULL	NULL

Figura 2: Tablas creadas en el visor del Database Provider de Supabase

4.2. Inserción de Datos Coherentes con Bloques Anónimos (PL/pgSQL)

Debido a que las llaves primarias se basan en identificadores de tipo UUID autogenerados, las inserciones manuales tradicionales resultan propensas a errores. Para garantizar la integridad referencial completa y poblar las tablas con datos del equipo de trabajo y las asignaturas del semestre, se diseñó y ejecutó el siguiente bloque anónimo:

```
DO $$
DECLARE
    admin_id UUID;
    gustavo_user_id UUID;
    geisel_user_id UUID;
    jesus_user_id UUID;

    gustavo_student_id UUID;
    geisel_student_id UUID;
    jesus_student_id UUID;

    curso_daw_id UUID;
    curso_eda_id UUID;
    curso_mn_id UUID;
BEGIN
    -- Insercin de un usuario Administrador/Docente para las auditoras
    INSERT INTO users (names) VALUES ('Ysabel Milagros Rodriguez Choque') RETURNING id INTO
    admin_id;

    -- Insercin de los perfiles en la tabla users
    INSERT INTO users (names) VALUES ('Gustavo Linares Aquino') RETURNING id INTO
    gustavo_user_id;
    INSERT INTO users (names) VALUES ('Geisel Reynar Pacheco Medina') RETURNING id INTO
    geisel_user_id;
    INSERT INTO users (names) VALUES ('Jesus Francisco Silva Pino') RETURNING id INTO
    jesus_user_id;

    -- Insercin correlativa en la tabla students, enlazando los user_id e incluyendo correos
    en note
    INSERT INTO students (names, "fatherSurname", "motherSurname", gender, address, phone,
    note, user_id, created_id)
    VALUES ('Gustavo', 'Linares', 'Aquino', 'Masculino', 'Av. Independencia 123, Arequipa',
    '987654321', 'Correo: glinares@unsa.edu.pe', gustavo_user_id, admin_id)
    RETURNING id INTO gustavo_student_id;
```

```
INSERT INTO students (names, "fatherSurname", "motherSurname", gender, address, phone,
note, user_id, created_id)
VALUES ('Geisel Reymer', 'Pacheco', 'Medina', 'Masculino', 'Av. Venezuela 456, Arequipa',
'912345678', 'Correo: gpachecome@unsa.edu.pe', geisel_user_id, admin_id)
RETURNING id INTO geisel_student_id;

INSERT INTO students (names, "fatherSurname", "motherSurname", gender, address, phone,
note, user_id, created_id)
VALUES ('Jesus Francisco', 'Silva', 'Pino', 'Masculino', 'Distrito de Cayma, Arequipa',
'998877665', 'Correo: jsilva@unsa.edu.pe', jesus_user_id, admin_id)
RETURNING id INTO jesus_student_id;

-- Insercin de Asignaturas del Semestre
INSERT INTO courses (name, description, created_id)
VALUES ('Desarrollo de Aplicaciones Web', 'Curso integrador de tecnologas web enfocado en
frontend y backend', admin_id)
RETURNING id INTO curso_daw_id;

INSERT INTO courses (name, description, created_id)
VALUES ('Estructura de Datos y Algoritmos', 'Anlisis, diseo y optimizacin de estructuras
de datos', admin_id)
RETURNING id INTO curso_eda_id;

INSERT INTO courses (name, description, created_id)
VALUES ('Mtodos Numricos', 'Resolucin de problemas matemticos mediante algoritmos',
admin_id)
RETURNING id INTO curso_mn_id;

-- Vinculacin de Matrculas en la tabla asociativa courses_students
INSERT INTO courses_students (course_id, student_id, created_id) VALUES
(curso_daw_id, jesus_student_id, admin_id),
(curso_daw_id, gustavo_student_id, admin_id),
(curso_daw_id, geisel_student_id, admin_id),
(curso_eda_id, jesus_student_id, admin_id),
(curso_mn_id, gustavo_student_id, admin_id),
(curso_mn_id, geisel_student_id, admin_id);
END $$;
```

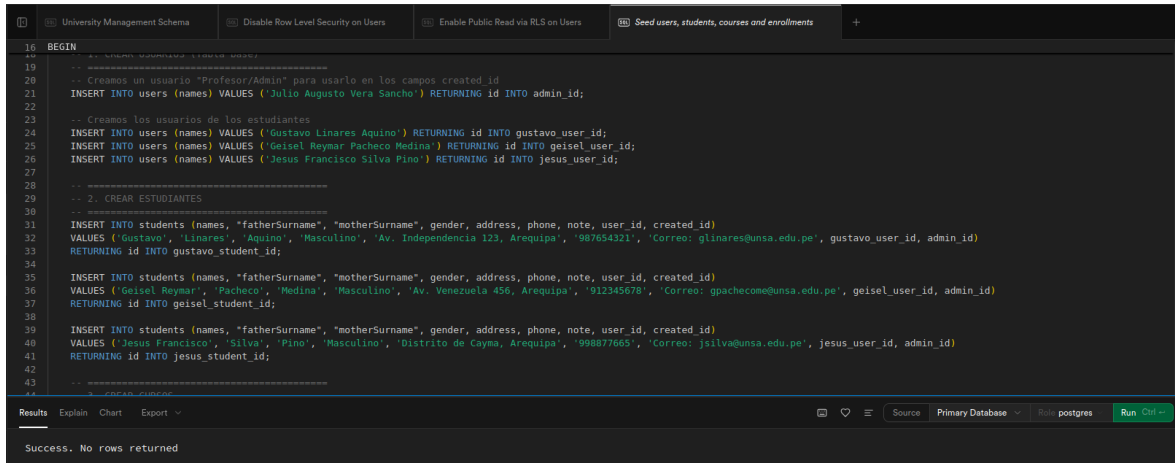


Figura 3: Captura de pantalla de Supabase con datos guardados exitosamente

5. Configuración de Seguridad: Row Level Security (RLS)

Durante las fases iniciales de prueba en Postman, a pesar de que los datos existían físicamente y la conexión devolvía un estado 200 OK, el cuerpo de la respuesta entregaba un arreglo vacío ([]). Esto se debe a que Supabase activa por defecto el aislamiento mediante Row Level Security (RLS) protegiendo el contenido contra lecturas no autorizadas.

Análisis Técnico: Para solucionar el aislamiento RLS se evaluaron dos alternativas. La primera opción consistía en deshabilitar por completo la seguridad con el comando `ALTER TABLE... DISABLE ROW LEVEL SECURITY`, lo cual es útil únicamente para entornos locales ágiles. Para este entregable, se optó por implementar la Opción 2, que representa las mejores prácticas de la industria, creando políticas explícitas que autorizan de manera segura el acceso público de lectura.

El script de seguridad aplicado a las cuatro tablas del laboratorio fue el siguiente:

```
-- Asegurar la activacin de la proteccin de filas en todas las entidades
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
ALTER TABLE students ENABLE ROW LEVEL SECURITY;
ALTER TABLE courses ENABLE ROW LEVEL SECURITY;
ALTER TABLE courses_students ENABLE ROW LEVEL SECURITY;

-- Establecer reglas que permitan el mtodo SELECT a clientes annimos autenticados por API Key
CREATE POLICY "Permitir lectura publica users" ON users FOR SELECT USING (true);
CREATE POLICY "Permitir lectura publica students" ON students FOR SELECT USING (true);
CREATE POLICY "Permitir lectura publica courses" ON courses FOR SELECT USING (true);
CREATE POLICY "Permitir lectura publica courses_students" ON courses_students FOR SELECT
    USING (true);
```

6. Integración y Pruebas en Postman

6.1. Configuración del Entorno de Trabajo (Environments)

Para consumir la API de Supabase de manera limpia, se configuró un entorno en Postman denominado **Supabase Local** con las siguientes variables globales, evitando tener que reescribir los tokens

e identificadores sensibles en cada endpoint:

- **base_url**: Contiene el punto de acceso API unificado de Supabase, concatenando la ruta de la versión de la API REST (Ejemplo: <https://tu-proyecto.supabase.co/rest/v1>).
- **api_key**: Almacena la clave pública de publicación o clave anon extraída desde el panel de configuraciones de la API de Supabase.

6.2. Inclusión Obligatoria de Cabeceras (Headers)

Cada petición enviada requiere de cabeceras HTTP específicas para interactuar con la puerta de enlace (Gateway) de Supabase:

Key (Cabecera)	Value (Valor asignado)	Propósito y Función Técnica
apikey	{{api_key}}	Autentica la petición frente al Gateway de API de Supabase.
Authorization	Bearer {{api_key}}	Pasa el token portador requerido por el middleware de seguridad.
Content-Type	application/json	Establece que el cuerpo de intercambio de datos utiliza formato JSON.

6.3. Validación de Respuestas Exitosas

Tras resolver las directivas RLS y de direccionamiento, las peticiones GET devolvieron códigos de estado HTTP 200 OK con la estructura de objetos JSON correspondientes a las tuplas de la base de datos de manera íntegra.

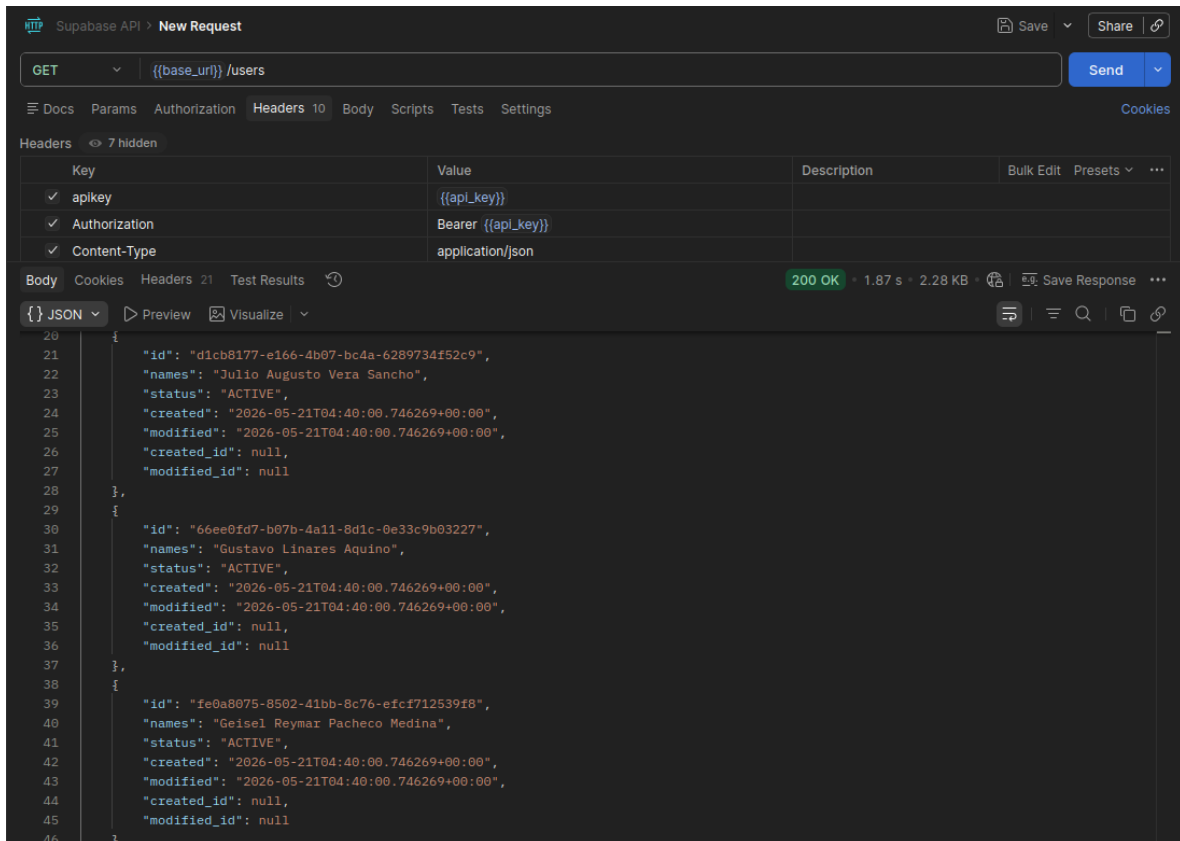


Figura 4: Postman ejecutando con éxito un GET a la tabla users”mostrando el JSON de respuesta

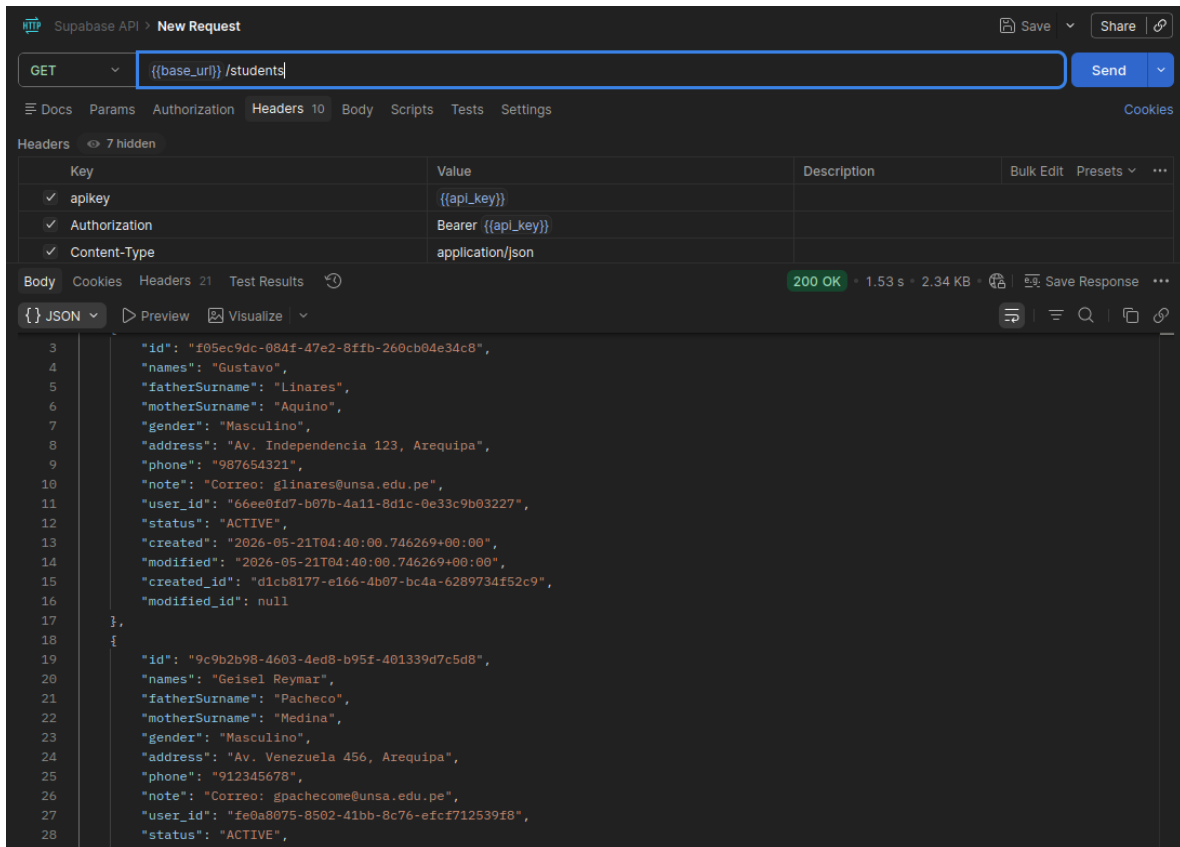


Figura 5: Postman ejecutando con éxito un GET a la tabla "students" mostrando los datos de contacto y notas de los integrantes

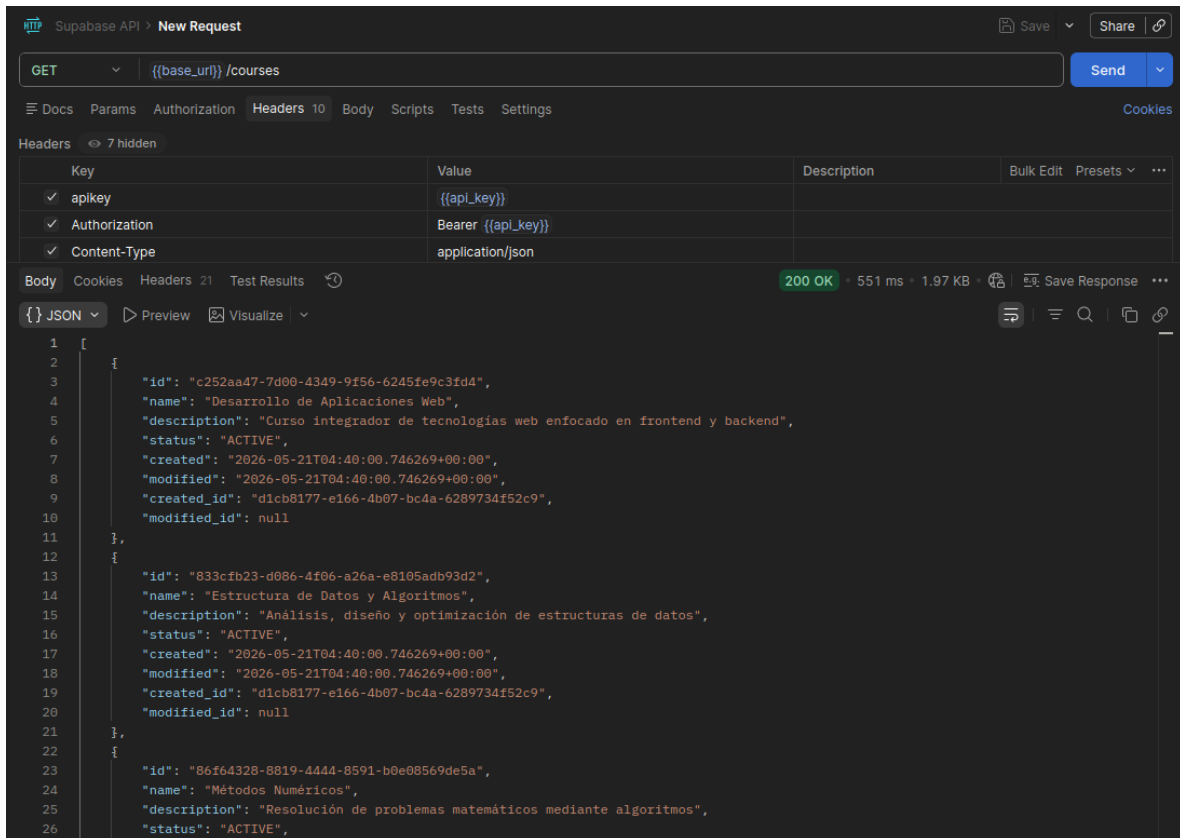


Figura 6: Postman ejecutando con éxito un GET a la tabla courses”mostrando el JSON de respuesta

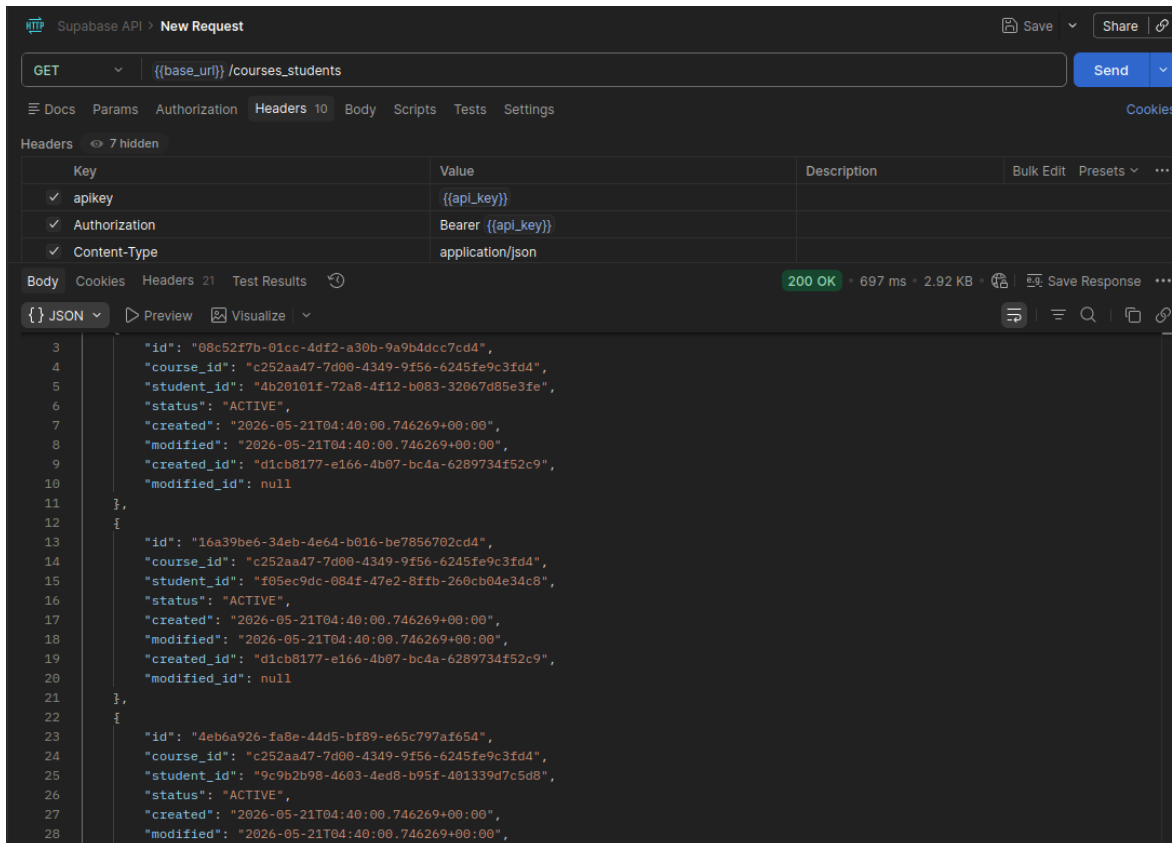


Figura 7: Postman ejecutando con éxito un GET a la tabla `courses_students` mostrando el JSON de respuesta

7. Conclusiones

- Las arquitecturas BaaS modernas como Supabase disminuyen drásticamente el tiempo de desarrollo de infraestructura web al proveer capas automáticas de API REST con documentación nativa a partir del modelo físico relacional.
- El uso de identificadores UUID en lugar de enteros autoincrementales incrementa la robustez de los sistemas distribuidos y previene vulnerabilidades de enumeración en endpoints públicos, requiriendo técnicas avanzadas de siembra como los bloques anónimos PL/pgSQL.
- La seguridad basada en Row Level Security (RLS) en entornos modernos de bases de datos es un pilar fundamental; delegar la seguridad directamente en el motor de base de datos a través de políticas condicionales (Policies) blindas las respuestas de las APIs integradas eliminando la dependencia exclusiva de capas lógicas externas en el backend.

8. Bibliografía y Referencias de Profundización

- PostgreSQL Global Development Group. (2026). PostgreSQL 16 Documentation: Chapter 5. Data Definition - Row Security Policies. Recuperado de <https://www.postgresql.org/docs/>

`current/ddl-rowsecurity.html`

- PostgreSQL Global Development Group. (2026). PostgreSQL 16 Documentation: SQL Commands - DO Statement. Recuperado de <https://www.postgresql.org/docs/current/sql-do.html>
- Supabase Inc. (2026). Supabase Developer Guides: Row Level Security & Access Control. Recuperado de <https://supabase.com/docs/guides/auth/row-level-security>
- Postman Incorporated. (2026). Postman Learning Center: Managing API Environments and Variables. Recuperado de <https://learning.postman.com/docs/sending-requests/managing-environments/>

9. Repositorio del Proyecto

- Repositorio GitHub del proyecto: <https://github.com/JesusFSP/Grupo-7---Laboratorio>

10. Rúbricas

10.1. Entregable Informe

Tabla 1: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio, buena presentación y fácil de leer.

10.2. Rúbrica para el contenido del Informe y demostración

Tabla 2: Rúbrica para contenido del Informe y demostración

Contenido y demostración	Descripción	Puntos	Checklist
GitHub	Hay enlace URL activo del directorio para el laboratorio hacia su repositorio GitHub con código fuente terminado y fácil de revisar.	2	X
Commits	Hay capturas de pantalla de los commits más importantes con sus explicaciones detalladas.	4	
Código fuente	Hay porciones de código fuente importantes con explicaciones de sus funciones.	2	X
Ejecución	Se incluyen ejecuciones y pruebas del código fuente explicadas gradualmente.	2	X
Pregunta	Se responde con completitud a las preguntas del cuestionario.	2	X
Fechas	Las fechas de modificación del código fuente están dentro de los plazos establecidos.	2	X
Ortografía	El documento no muestra errores ortográficos.	2	X
Madurez	El informe muestra evolución del código fuente, explicaciones puntuales y un acabado limpio.	4	X
Total		20	